

Week 5, Part 3: Forms and Validation

Dr M AOUE

Lecture notes

Objectives. This is the third part of week 5. By the end you will be able to let the user pick one value from a list with `DropDownButton`; group several fields in a `Form` and reach it from outside with a `GlobalKey<FormState>` or with `Form.of(context)`; validate a `TextFormField` and show its own error message; run every field's validator at once with `validate()`, and clear a form with `reset()`; compose several small validator functions instead of one large one; restrict what can be typed with `inputFormatters`; react to every change of a controller with `addListener`; and write your own `FormField<T>` around a custom input widget.

The **programs** of this part are not printed here: they are in the project **examples**, inside `code/week05` (section 7), the same project as part 2. Each section quotes only the few lines that matter. Every program starts with a comment that says what it teaches and what result to expect, and automated tests check that it does what is described. Parts 1 and 2 of the week, *Creating Interactive Applications* and *Gestures and State*, are separate documents; state management itself (ephemeral state, lifting state up, `ChangeNotifier`, the `provider` package) was covered in part 2 and is not repeated here.

1 The `DropDownButton` widget

`DropDownButton<T>` lets the user pick **one** value out of a fixed list, shown as a menu when the button is tapped.

Field	Meaning
<code>items</code>	a <code>List<DropDownMenuItem<T>></code> : one entry per choice
<code>value</code>	the <i>currently</i> selected value; it must be one of <code>items</code> ' values, or <code>null</code>
<code>onChanged</code>	called with the new value when the user picks one
<code>iconSize</code>	the size of the small arrow on the right

You rarely write the `items` by hand. You build them from a list of data with `.map(...).toList()`: one `DropDownMenuItem<String>` is created for each string of the list.

```
1 final List<String> _cities = ['Calcutta', 'Delhi', 'Mumbai', 'Chennai', '
   Bangalore'];
2 ...
3 items: _cities.map((city) {
4   return DropDownMenuItem<String>(value: city, child: Text(city));
5 }).toList(),
```

`DropDownButton` is **stateless by itself**: it does not remember what is selected. *You* keep the selected value in the page's state and give it back as `value`; `onChanged` only *reports* the user's choice, and `setState` stores it and redraws the button showing the new value. This is the same “data down, events up” idea as the callback of a custom widget (week 5, part 1).

```

1 void _selectCity(String? city) {
2   setState(() {
3     if (city != null) _selectedCity = city;
4   });
5 }
6 ...
7 DropdownButton<String>(<
8   value: _selectedCity,
9   iconSize: 32,
10  items: _cities.map((city) => DropdownMenuItem(value: city, child: Text(city)))
11    .toList(),
12  onChanged: _selectCity,
13),

```

The callback's parameter is nullable, `ValueChanged<String?>`. Writing `onChanged: (String city) { ...}` (no `?`), as some older tutorials do, does *not* compile: we checked, and the compiler refuses it (*argument type "void Function(String)" can't be assigned to the parameter type "ValueChanged<String?>?"*). A drop-down can genuinely have nothing selected, so Flutter's API allows null.

`initState` sets the first value, once, before the first build:

```

1 late String _selectedCity;
2
3 @override
4 void initState() {
5   super.initState();
6   _selectedCity = _cities[0]; // the first item is selected at the start
7 }

```

Without it, `_selectedCity` would start undefined, and a value that is not null and not one of items' values makes `DropdownButton` show nothing selected (or, under late, throw before the first frame if never assigned).

2 Grouping fields: Form and GlobalKey

A page often has several fields that must be checked *together* before anything is submitted. The `Form` widget groups them; it draws nothing itself, it only **coordinates its fields**.

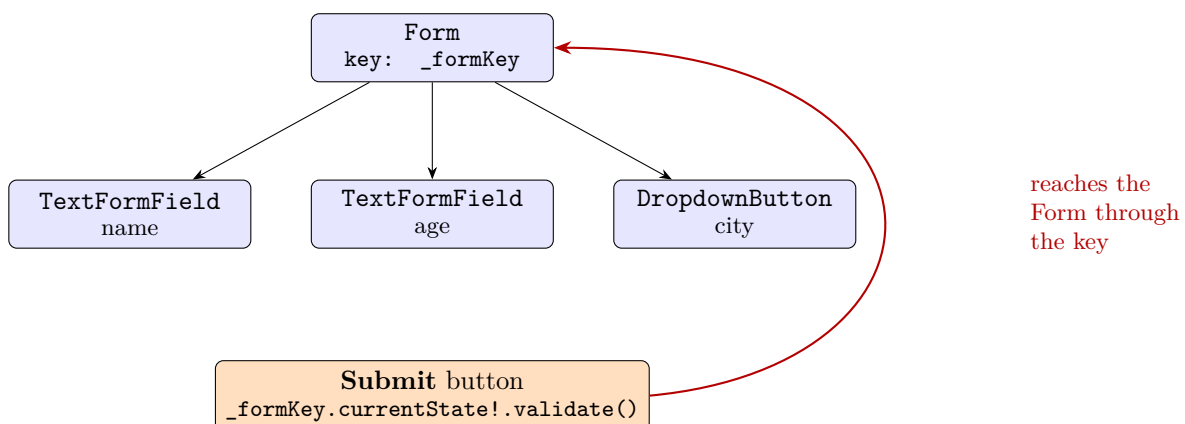


Figure 1: A `GlobalKey<FormState>` lets code *outside* the `Form` (the `Submit` button) reach its state and validate every field at once.

The key

We give the Form a `GlobalKey`, a special key that lets code **outside** the widget tree reach that widget's state:

```
1 final _formKey = GlobalKey<FormState>();
2 ...
3 Form(
4   key: _formKey,
5   child: ListView(children: [ ... the fields ... ]),
6 ),
```

`_formKey.currentState` is the `FormState` object, with three useful methods: `validate()` (below), `save()` (calls every field's `onSaved`), and `reset()` (clears every field and its errors).

TextFormField and validator

`TextFormField` is a `TextField` that also plugs into a `Form`: it can be **validated**, and it shows its own error message under the field. Its `validator` field is a function that receives the current text and returns:

Return value	Meaning
<code>null</code>	the text is fine: no error is shown
a <code>String</code>	that string is shown in red under the field

```
1 String? _requiredValidator(String? value) {
2   if (value == null || value.trim().isEmpty) {
3     return 'Please fill in this field';
4   }
5   return null; // null means: no error
6 }
```

The signature is `String? Function(String?)`: both the parameter and the return type are nullable. Declaring the parameter as a plain `String` (not `String?`), as older code sometimes does, no longer compiles: we checked, and it is a real type error ("*String? Function(String) can't be assigned to ... TextFormFieldValidator<String>?*"), not a style nitpick. Ending the function without an explicit `return null`; at the end is only a warning, not an error: Dart returns `null` anyway if a nullable function falls off the end, but writing it makes the intent clear.

Validating: `currentState!.validate()`

Pressing **Submit** calls `_formKey.currentState!.validate()`. This runs *every* field's validator, shows the error messages, and returns `true` only if they *all* returned `null`.

```
1 void _submit() {
2   if (_formKey.currentState!.validate()) {
3     setState(() {
4       _result = 'Name: ${_nameController.text}, age: ${_ageController.text},
5         city: $_selectedCity';
6     });
7   }
8 }
```

`currentState` is **nullable** (`FormState?`): the `Form` might not be built yet the first time the key is used. Calling `.validate()` without the `!`, as older code does, does not compile: we checked,

it is a real error (*“The method ‘validate’ can’t be unconditionally invoked because the receiver can be ‘null’”*).

Resetting the form

`reset()` clears the *validation errors*; it does not touch the text of the fields by itself in this example, because the fields use a `TextEditingController` (week 5, part 1) rather than `initialValue`, so we clear the controllers ourselves, then update our own state:

```
1 void _reset() {
2   _formKey.currentState!.reset(); // clears the validation errors
3   _nameController.clear();
4   _ageController.clear();
5   setState(() {
6     _selectedCity = _cities[0];
7     _result = '';
8   });
9 }
```

Putting it together

Example 11_form_validation combines two `TextFormFields`, a `DropDownButton`, and the two buttons:

```
1 Form(
2   key: _formKey,
3   child: ListView(
4     children: [
5       TextFormField(
6         controller: _nameController,
7         decoration: const InputDecoration(labelText: 'Your name', border:
8         OutlineInputBorder()),
9         validator: _requiredValidator,
10      ),
11      const SizedBox(height: 12),
12      TextFormField(
13        controller: _ageController,
14        keyboardType: TextInputType.number,
15        decoration: const InputDecoration(labelText: 'Your age', border:
16        OutlineInputBorder()),
17        validator: _requiredValidator,
18      ),
19      const SizedBox(height: 12),
20      DropDownButton<String>(
21        value: _selectedCity,
22        items: _cities.map((city) => DropdownMenuItem(value: city, child: Text(
23        city))).toList(),
24        onChanged: (city) => setState(() { if (city != null) _selectedCity =
25        city; }),
26      ),
27      ... // the Submit and Reset buttons, and the result Text
28    ],
29  ),
30 ),
```

3 Going further: composing validators

A validator is a normal Dart function: any logic is allowed. Instead of writing one large function per field, example 12_more_validators writes several **small** validators and **chains** them:

```

1 String? _validEmail(String? value) {
2   final pattern = RegExp(r'^[^\s]+@[^\s]+\.[^\s]+$');
3   if (value != null && !pattern.hasMatch(value)) {
4     return 'Not a valid email address';
5   }
6   return null;
7 }
8
9 String? Function(String?) _minLength(int length) {
10  // Returns a validator function: a function that BUILDS a function.
11  return (value) {
12    if (value != null && value.length < length) return 'At least $length
13    characters';
14    return null;
15  };
16 }
17
18 // Runs several validators in order and stops at the first error.
19 String? _combine(List<String? Function(String?)> validators, String? value) {
20   for (final validator in validators) {
21     final error = validator(value);
22     if (error != null) return error;
23   }
24   return null;
25 }

```

Each field then lists the checks it needs, in order:

```

1 validator: (v) => _combine([_required, _validEmail], v),
2 ...
3 validator: (v) => _combine([_required, _minLength(6)], v),

```

- `_minLength` is a function that *returns* a function: calling `_minLength(6)` does not check anything yet, it *builds* a validator that will check for 6 characters when it is later called with a value. This is the same idea as the callback of week 5, part 1, one level up.
- A validator can also depend on *another* field, for a “confirm password” check: read the other controller’s `.text` inside the function.

```

1 String? _confirmValidator(String? value) {
2   if (value != _passwordController.text) return 'Passwords do not match';
3   return null;
4 }

```

When to validate: `autovalidateMode`

By default a field validates only when `.validate()` is called. The Form’s `autovalidateMode` field changes this:

AutovalidateMode	Validates
disabled	only when <code>.validate()</code> is called (the default; example 11_form_validation uses it)
onUserInteraction	after the user first touches a field, then on every change of that field
always	from the very first frame, even before the user has typed anything

`onUserInteraction` gives feedback as the user types, without shaming an empty field before they have even started. Example 12_more_validators uses it:

```

1 Form(
2   key: _formKey,
3   autovalidateMode: AutovalidateMode.onUserInteraction,
4   child: ListView(children: [ ... ]),
5 ),

```

4 Going deeper into forms

The four sections below extend the forms topic with material used to build real, custom inputs: restricting what the user can type, reacting to every change of a controller (not only the user's), a second way to reach a `Form`'s state, and finally writing your own `FormField`.

Restricting what can be typed: `inputFormatters`

A **validator** rejects a *wrong* value **after** it has been typed. An **inputFormatter** goes further: it stops the wrong character from ever *appearing* in the field. The two are complementary: use formatters to guide typing, and a validator to check the final result (for example “not empty”, which a formatter cannot express).

`TextField` (and `TextFormField`) take a list of `inputFormatters`, run in order:

```

1 TextField(
2   keyboardType: TextInputType.number,
3   inputFormatters: [
4     FilteringTextInputFormatter.digitsOnly, // only 0-9
5     LengthLimitingTextInputFormatter(6),   // at most 6 characters
6   ],
7   ...
8 ),

```

Formatter	Effect
<code>FilteringTextInputFormatter.allow(pattern)</code>	keeps only the characters that <i>match</i> pattern (a <code>RegExp</code>)
<code>FilteringTextInputFormatter.deny(pattern)</code>	removes the characters that match
<code>FilteringTextInputFormatter.digitsOnly</code>	a ready-made shortcut for <code>.allow(RegExp(r'[0-9]'))</code>
<code>LengthLimitingTextInputFormatter(n)</code>	stops the field at <code>n</code> characters

Example 13_input_formatters shows this on its own: `keyboardType: TextInputType.number` is only a *hint* for the on-screen keyboard (week 5, part 1); a physical keyboard can still type letters, which is why the formatter is still needed to truly block them.

Listening to a controller: `addListener` versus `onChanged`

Week 5, part 1 showed that when the *program* changes a field's text (`controller.text = '...'`), `onChanged` is **not** called: only the user's own typing triggers it. `TextEditingController` is a `ValueNotifier`, and setting `.text` calls `notifyListeners()` *whichever way* the text changed, by the user or by the program. So a listener added with `controller.addListener(...)` fires in **both** cases.

```

1 _controller.addListener(_onControllerChanged);
2 ...
3 // No parameter: the new text is read from the controller itself.
4 void _onControllerChanged() {
5   setState(() {

```

```

6   _mirrored = _controller.text;
7   });
8 }

```

Use...	When
onChanged	you only care about what the <i>user</i> typed (most of week 5, part 1)
controller.addListener(...)	you need to react to <i>every</i> change of the text, including ones made by your own code

Like any resource you set up, remove the listener and dispose the controller in `dispose`: forgetting `removeListener` leaks the callback, forgetting `dispose` leaks the controller (week 5, part 1). Example 14_controller_listener shows both a listener that mirrors the user's typing and a button that changes the text *programmatically*, and the listener catches both.

Reaching a Form without a key: Form.of(context)

Example 11_form_validation reaches a Form's state with a GlobalKey. There is a second way: `Form.of(context)`, which walks **up** the widget tree from `context` and returns the nearest Form's state. It works like `Theme.of(context)` or `MediaQuery.of(context)` (week 4): `Form` exposes itself to its descendants through an `InheritedWidget`, and `.of()` is the standard way Flutter widgets do this.

context must belong to a widget that is below the Form, not the one that *built* it. The context of the build method that creates the Form is still *above* it: `Form.of(context)` there fails. A Builder widget solves this: its own builder function receives a *new*, inner BuildContext, for the point right where the Builder sits, which is below the Form.

```

Form(
  child: Column(
    children: [
      TextFormField validator: ...,
      // A Builder gives its child a context BELOW the Form.
      Builder(
        builder: (subContext) {
          return ElevatedButton(
            onPressed: () => Form.of(subContext).validate(),
            child: const Text('VALIDATE'),
          );
        },
      ),
    ],
  ),
),

```

`Form.of(context)` is **non-nullable**: if no Form is found, it throws (we checked the framework source: it asserts in debug mode and throws a `FlutterError`, “*Form.of() was called with a context that does not contain a Form widget*”). `Form.maybeOf(context)` returns `null` instead, if you want to handle the “no Form” case yourself.

Which one to use? A `GlobalKey` is simpler when the button is a direct sibling of the `Form`, in the same `build` method. `Form.of(context)` is convenient when the button is written far from the `Form`, for example inside a separate widget class that never receives a key.

Creating your own FormField

TextFormField is not magic: it is a `FormField<String>` that happens to wrap a `TextField`. You can build your **own** `FormField<T>` around any input widget, and it gets `validate()`, `reset()` and `save()` for free, exactly like a built-in one. Example `16_custom_formfield` builds a 6-digit verification-code field, in two parts.

Part 1: a plain input widget.

It knows nothing about Form: the same widget could be used outside any form. It reuses the formatters of the previous section.

```
1 class VerificationCodeInput extends StatelessWidget {
2   const VerificationCodeInput({super.key, required this.controller, this.
    errorText, this.onChanged});
3   final TextEditingController controller;
4   final String? errorText;
5   final ValueChanged<String>? onChanged;
6
7   @override
8   Widget build(BuildContext context) {
9     return TextField(
10      controller: controller,
11      inputFormatters: [FilteringTextInputFormatter.digitsOnly,
12      LengthLimitingTextInputFormatter(6)],
13      decoration: InputDecoration(errorText: errorText),
14      onChanged: onChanged,
15    );
16  }
```

Part 2: a `FormField<String>` that builds it.

A `FormField<T>` is created with a builder function, which receives a `FormFieldState<T>` and returns the widget to show. The state has `field.value`, `field.errorText` and `field.didChange(newValue)`: calling `didChange` tells the Form “the value changed”, which is what makes `validate()` see the current text.

To add a **persistent controller** to the field, so it survives rebuilds and can be cleared by `reset()`, the `FormField` subclass overrides `createState()` to return its *own* state class, which must extend `FormFieldState<T>` (**not** the plain `State<T>` of an ordinary widget):

```
1 class VerificationCodeFormField extends FormField<String> {
2   VerificationCodeFormField({super.key, super.onSaved, super.validator, super.
    autovalidateMode})
3     : super(
4       initialValue: '',
5       builder: (field) {
6         final state = field as _VerificationCodeFormFieldState;
7         return VerificationCodeInput(
8           controller: state._controller,
9           errorText: field.errorText,
10          onChanged: field.didChange,
11        );
12      },
13    );
14
15   @override
16   FormFieldState<String> createState() => _VerificationCodeFormFieldState();
17 }
18
19 class _VerificationCodeFormFieldState extends FormFieldState<String> {
20   final TextEditingController _controller = TextEditingController();
21 }
```

```

22 @override
23 void initState() {
24   super.initState();
25   _controller.addListener(_handleControllerChanged);
26 }
27
28 void _handleControllerChanged() {
29   if (_controller.text != value) didChange(_controller.text);
30 }
31
32 @override
33 void reset() {
34   super.reset(); // resets the Form's value to initialValue (='')
35   _controller.text = ''; // and clears what is actually shown
36 }
37
38 @override
39 void dispose() {
40   _controller.removeListener(_handleControllerChanged);
41   _controller.dispose();
42   super.dispose();
43 }
44 }

```

- At runtime, the field the builder receives *is* a `_VerificationCodeFormFieldState`, because `createState()` returns one: the cast is safe.
- The controller's listener calls `didChange(controller.text)`, so the Form always has the controller's latest text, whether it changed from typing or from code.
- `reset()` must call `super.reset()` *and* clear the controller: the first resets the Form's notion of the value, the second clears what is actually drawn on screen.

The custom field is then used exactly like a `TextFormField`:

```

1 Form(
2   key: _formKey,
3   child: Column(
4     children: [
5       VerificationCodeFormField(
6         validator: _validateCode,
7         onSave: (value) => setState(() => _accepted = 'Code accepted: $value'),
8       ),
9       ElevatedButton(
10        onPressed: () {
11          if (_formKey.currentState!.validate()) {
12            _formKey.currentState!.save(); // calls onSave with the current
13            value
14            _formKey.currentState!.reset();
15          }
16        },
17        child: const Text('SUBMIT'),
18      ),
19    ],
20  ),

```

FormState has no way to *read* a field's value directly (a Form only coordinates its fields, it does not collect their values): call `validate()`, then `save()` to *collect* each field's value through its `onSaved` callback, as in the code above.

5 What we changed from the printed text

The chapter that this part follows was written for an older version of Flutter, before sound null safety. The programs of this course use the current API, and we **verified** each correction below by compiling the old form (with `flutter analyze`) and reading the real error:

In the printed text	In our programs, and why
<code>validator: (String validationValue) { ...}</code>	<code>validator: (String? value) { ...}</code> : the parameter must be nullable. <i>Verified error: argument type “String? Function(String)” can’t be assigned to ... FormFieldValidator<String>?</i>
<code>if (_formKey.currentState.validate())</code>	<code>if (_formKey.currentState!.validate()):</code> <code>currentState is FormState?.</code> <i>Verified error: The method ‘validate’ can’t be unconditionally invoked because the receiver can be ‘null’</i>
<code>onChanged: (String theValueSelected) { ...}</code>	<code>onChanged: (String? city) { ...}</code> (or a method with a nullable parameter): the same nullability rule applies to <code>DropDownButton</code> . <i>Verified error: argument type “void Function(String)” can’t be assigned to ... ValueChanged<String>?</i>
<code>RaisedButton, textColor, color</code>	<code>ElevatedButton / OutlinedButton,</code> <code>foregroundColor, backgroundColor</code> (week 4, part 3; week 5, part 1)
<code>int age = int.parse(textOfAge.text)</code>	the same crash risk as <code>double.parse</code> (week 5, part 1): a <code>validator</code> that checks the text is a number should run <i>before</i> <code>int.parse</code> is used, or <code>int.tryParse</code> should be used instead

6 Common errors

Message or symptom	Cause	Fix
<i>The method ‘validate’ can’t be unconditionally invoked because the receiver can be ‘null’</i>	<code>currentState</code> used without <code>!</code> or a null check	<code>_formKey.currentState!.validate()</code>
<i>argument type ... can’t be assigned to ...</i> <i>FormFieldValidator<String>?</i>	<code>validator</code> ’s parameter is not nullable	write <code>(String? value)</code>
No error appears when the field is empty	the field is a plain <code>TextField</code> , not a <code>TextFormField</code> , so it has no <code>validator</code> to call	change it to <code>TextFormField</code>
Errors appear on a page with no Submit button	<code>autovalidateMode</code> is <code>always</code> (or the field was touched)	this may be intended; use <code>disabled</code> to validate only on <code>.validate()</code>

FormatException after Submit	<code>int.parse</code> / <code>double.parse</code> runs on text that the validator allowed through as “not empty” but that is not a number	add a number-format check to the validator, or use <code>tryParse</code>
The dropdown shows nothing selected	<code>value</code> is not <code>null</code> and not one of <code>items</code> ’ values (for example <code>initState</code> was not run, or the list changed)	set <code>value</code> to <code>null</code> or to a value that is really in <code>items</code>
<i>Form.of()</i> was called with a context that does not contain a Form widget	the <code>context</code> used belongs to the <code>build</code> method that created the <code>Form</code> (still above it), not a widget below it	wrap the caller in a <code>Builder</code> and use its <code>context</code> , or use a <code>GlobalKey</code> instead
A custom <code>FormField</code> ’s value never reaches <code>validate()</code>	the input widget’s changes are not reported with <code>field.didChange(...)</code>	call <code>didChange</code> in the controller’s listener, or in the input’s <code>onChanged</code>

7 The programs

The code of this part is **not** printed in this document. It is in the folder `code/week05/examples`, the same project as part 2 (`provider` is already a dependency there, but these three examples do not use it). See the guide *Running the Flutter apps in VS Code* to prepare the tools; then, from the folder `examples`:

```
flutter pub get
flutter test
flutter create .                (one time only)
flutter run -t lib/11_form_validation.dart
```

File	What you see and try
10_dropdown_button	a text and a dropdown of five cities; picking one updates the text
11_form_validation	two <code>TextFormFields</code> , a dropdown, SUBMIT and RESET ; empty fields show red errors, a valid form shows a result, RESET clears everything
12_more_validators	email, password and confirm-password fields, checked live as you type, with chained validators
13_input_formatters	a 6-digit code field: letters are refused, and typing stops at 6 digits
14_controller_listener	a field mirrored by <code>addListener</code> ; a button changes it <i>programmatically</i> , and the mirror still follows
15_form_of_builder	the same validation as example 11, reached with <code>Form.of(context)</code> through a <code>Builder</code> instead of a <code>GlobalKey</code>
16_custom_formfield	a hand-written <code>VerificationCodeFormField</code> , used inside a <code>Form</code> exactly like a <code>TextFormField</code>

8 Exercises

1. In `10_dropdown_button`, add a sixth city, and make `value` start at `null` instead of `_cities[0]`: what does the dropdown show before a choice is made?
2. In `11_form_validation`, add a third field, "Email", with the `_validEmail` check of example 12.
3. In `11_form_validation`, change the age field's validator so that it also rejects a text that is not a whole number (use `int.tryParse`).
4. In `11_form_validation`, remove the `!` after `currentState` and read the error the analyzer gives.
5. In `12_more_validators`, add a fourth validator, `_noSpaces`, that rejects a value containing a space, and chain it into the email field.
6. Change the `autovalidateMode` of `12_more_validators` to `AutovalidateMode.always`: what changes on the very first frame, before you touch anything?
7. Add an `onSaved` to a field of `11_form_validation`, and call `_formKey.currentState!.save()` in `_submit`, before reading the controllers.
8. In `13_input_formatters`, replace `digitsOnly` with `FilteringTextInputFormatter.allow` and a pattern for letters instead of digits: what can you type now?
9. In `14_controller_listener`, add a second button, **CLEAR**, that sets `_controller.text` to `"`: does the mirrored line follow?
10. In `15_form_of_builder`, remove the `Builder` and call `Form.of(topContext)` directly from the outer `build` method: read the error.
11. In `16_custom_formfield`, add a **RESET** button next to **SUBMIT** that calls `reset()` on the form key directly, without validating first.
12. In `16_custom_formfield`, change `_validateCode` to also refuse the code `"000000"` (a placeholder value), and test it.

9 Summary

Idea	In one sentence
<code>DropDownButton<T></code>	items (built with <code>.map().toList()</code>), value (kept in your state), <code>onChanged</code> (a nullable callback)
<code>Form</code>	groups fields so they can be validated and reset together; draws nothing itself
<code>GlobalKey<FormState></code>	lets code outside the <code>Form</code> reach it: <code>.currentState!.validate()</code> / <code>.save()</code> / <code>.reset()</code>
<code>TextFormField</code>	a <code>TextField</code> that can be validated; its <code>validator</code> shows its own error
<code>validator</code>	<code>String? Function(String?)</code> : null means no error, a <code>String</code> is the error shown
<code>validate()</code>	runs every field's validator; <code>true</code> only if they all returned null
<code>Composing validators</code>	small functions, one check each, run in order by a <code>_combine</code> helper
<code>autovalidateMode</code>	<code>disabled</code> (only on <code>.validate()</code>), <code>onUserInteraction</code> , always
<code>inputFormatters</code>	filter or limit what can be <i>typed</i> , before any validator runs: <code>FilteringTextInputFormatter</code> , <code>LengthLimitingTextInputFormatter</code>
<code>controller.addListener</code>	fires for every change of the text, by the user <i>or</i> the program; <code>onChanged</code> only fires for the user
<code>Form.of(context)</code>	a second way to reach a <code>Form</code> 's state; <code>context</code> must be <i>below</i> the <code>Form</code> (use a <code>Builder</code>)
<code>Custom TextFormField<T></code>	a builder plus a state that extends <code>FormFieldState<T></code> (not plain <code>State<T></code>), calling <code>didChange</code> to notify the <code>Form</code>

This document is maintained by Dr. Mohamad AOUDÉ
Last updated: 2026-09-22